

Document Smart Highlights

1 Version

0.3.0-SNAPSHOT - RC21 - 20260927-025432

2 Introduction

Welcome to **DSH - Document Smart Highlights**. Document Smart Highlights aims to provide a set of web services to allow uploading of PDF and HTML files and return a list of keywords and most relevant sentences. This system applies state of the art algorithms, using NLP techniques to produce both the list of keywords and most relevant sentences. This last one, using automatic document summarization techniques. More details at the [wiki](#).

This project is distributed as source code. In order to generate the application to run it is needed to install the pre-requisites and compile. No binaries distribution is provided for now. The final application is a java .WAR file which can be dropped in a standard java servlet container.

The goals for this project are basically two:

1. Implement a NLP based system to extract relevant information from PDF files in a potentially scalable fashion by using NoSql databases and Queues. This infrastructure stores requests and chain a workflow of operations (workers) which will do the operations of keyword and relevant sentences extraction. All provided as a web service REST API.
2. Learn concepts of NLP associated with scaleable cloud based REST API building. The technology used is java based, so as another goal here we can mention the build learning process using [Spring Framework](#) in order to build the API and infrastructure.

As part of the goals is learning about technologies, comments and contributions are welcome. However, this is not a final product, application or concept. Just a point for experimentation and proofing.

If anyone out there is interested in contribute or apply this project on a more product-oriented environment, please get in touch through the email: marcelo.riss@gmail.com.

Wiki: <https://github.com/MRISS-Projects/dsh/wiki>

Project Development Documentation: <https://mriss-projects.github.io/dsh-docs/>

3 Package/Folders Description

- **dsh-data**: data models definition for a **Document**, **Keyword** and **RelevantSentence**. Additionally this module defines a [workflow](#) to map the status progress of a document processing request.
- **dsh-doc-analyzer**: This is a container module to have the keyword and relevant sentences extractor modules. One extra module to dequeue documents to be analyzed and call the extractors.
 - **dsh-doc-processor-worker**: The keyword extraction is executed by setting scores to each term in the document's text.
 - **dsh-keyword-extractor**: The keyword extraction is executed by setting scores to each term in the document's text.
 - **dsh-top-sentences-extractor**: The extraction of top sentences is achieved applying typical [automatic summarization](#) techniques like extractive summarization or [key phrase extraction](#).

- **dsh-doc-indexer-worker**: dequeues a document id from a queue, gets the document from the database and send it for indexing at **SOLR**. Besides indexing, this module will extract the text, paragraphs, sentences on each paragraph and terms on each sentence.
- **dsh-rest-api**: this module is the real application to be deployed or installed on a servlet container. It is the entry point for document submission, document processing status querying and document processing results (keywords and relevant sentences) querying.
- **dsh-test-dataset**: default and common data set of files used for automated testing.

4 Installation

4.1 Pre-requisites

- Java 17 (Temurin)
- Maven 3.9.16
- MongoDB 3.4 (windows 10)
- MongoDB 4.0.6 (Ubuntu 18.04 LTS)
- RabbitMQ 3.6.14 (windows 10)
- RabbitMQ 3.7.14 (Ubuntu 18.04 LTS)
- Tomcat 8.0.X

4.2 Installing/Building the Application

4.2.1 Java

Download and Installation

1. Download an Eclipse Temurin JDK 17 from <https://adoptium.net/temurin/releases/?version=17>.

1. **IMPORTANT NOTE**: Download and install a **JDK**, **not a JRE**. On the download page select Version **17 (LTS)**, your operating system and architecture, and the **JDK** package type.

2. Windows

1. There should be a .msi windows installer. Just follow the instructions.

3. Linux

1. Download the .tar.gz file. After downloading it, uncompress it at a folder of your preference.
2. Create a link. Open a command prompt, go to the JDK parent folder (the folder where you extract JDK into), and type:

```
# replace jdk-17.0.20.1+1 with the directory the archive extracted to
ln -s jdk-17.0.20.1+1 java
```

Setting environment variables

Linux

1. Open the file /home/[YOUR_USER]/.profile. This file might be hidden. If it does not appear at your home folder, using the file explorer, type Ctrl+h. Go to the end of the file and add:

```
export JAVA_HOME=/your/jdk/parent/folder/java
export PATH=$JAVA_HOME/bin:$PATH
```

Windows

1. Open Control Panel go to System, Advanced system settings, Environment Variables button.
2. At the System Variables section, click New.
3. Set JAVA_HOME and point to the root of JDK folder.
4. Search for the variable named «Path» in the list, click on it and press Edit.
5. Prepend the value with:

```
%JAVA_HOME%\bin;
```

Verifying the installation

1. Open a command prompt and type:

```
java -version
```

2. The result should be something like:

```
openjdk version "17.0.20.1" 2026-08-18
OpenJDK Runtime Environment Temurin-17.0.20.1+1 (build 17.0.20.1+1)
OpenJDK 64-Bit Server VM Temurin-17.0.20.1+1 (build 17.0.20.1+1, mixed mode, sha
```

4.2.2 Maven

1. Download maven **3.9.16** from <https://archive.apache.org/dist/maven/maven-3/3.9.16/binaries/apache-maven-3.9.16-bin.zip>
2. Unzip it on a folder of your preference
3. Set environment variables.

1. Linux

1. Put it at your \$HOME/.profile file

```
export M2_HOME=/path/to/where/you/extracted/maven/apache-maven-3.9.16
export PATH=$M2_HOME/bin:$PATH
export MAVEN_OPTS=' -Xmx1024m '
```

2. If you already have java set up, your .profile, it should look like this:

```
export JAVA_HOME=/your/jdk/parent/folder/java
export M2_HOME=/path/to/where/you/extracted/maven/apache-maven-3.9.16
export PATH=$JAVA_HOME/bin:$M2_HOME/bin:$PATH
export MAVEN_OPTS='-Xmx1024m'
```

3. Logout and login again.
4. Test by opening a terminal and typing:

```
mvn -version
```

5. The result should be similar to:

```
Apache Maven 3.9.16 (2bdd9fddda4b155ebf8000e807eb73fd829a51d5)
Maven home: /home/[YOUR_USER]/apps/apache-maven-3.9.16
Java version: 17.0.20.1, vendor: Eclipse Adoptium, runtime: /home/[YOUR_
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "6.8.0-45-generic", arch: "amd64", family: "u
```

2. Windows

1. Open Control Panel go to System, Advanced system settings, **Environment Variables** button.
2. At the System Variables section, click New.
3. Set M2_HOME and point to the root of maven folder.
4. Search for the variable Path in the list, click on it and press Edit.
5. Put the maven bin folder right after java home:

```
%JAVA_HOME%\bin;%M2_HOME%\bin;
```

6. Set the variable MAVEN_OPTS.

```
MAVEN_OPTS=-Xmx1024m
```

7. The result should be similar to:

```
Apache Maven 3.9.16 (2bdd9fddda4b155ebf8000e807eb73fd829a51d5)
Maven home: C:\data\apache-maven-3.9.16
Java version: 17.0.20.1, vendor: Eclipse Adoptium, runtime: C:\Program F
Default locale: en_US, platform encoding: Cp1252
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"
```

4.2.3 MongoDB

Windows

1. Install MongoDB using the instructions at [this link](#)
2. Enable security following general guidelines at [this link](#)
3. Start MongoDB:

```
"C:\Program Files\MongoDB\Server\3.4\bin\mongod.exe"
```

4. In another prompt connect to MongoDB

```
"C:\Program Files\MongoDB\Server\3.4\bin\mongo.exe"
```

5. Create super user

```
$ use admin
$ db.createUser(
{
  user: "superAdmin",
  pwd: "[your admin password]",
  roles: [ { role: "root", db: "admin" } ]
})
```

6. Disconnect and re-connect at MongoDB as super user:

```
connect-mongo-super-user.bat [your admin password]
```

7. Create user access (readWrite) for specific dsh database

```
$ use dsh
$ db.createUser(
{
  user: "dshuser",
  pwd: "[your password]",
  roles: [ "readWrite" ]
})
```

8. Disconnect and re-connect at MongoDB as specific user:

```
connect-mongo.bat [your dshuser password]
```

Linux Ubuntu 18.04 LTS

1. Install MongoDB following the instructions at <https://docs.mongodb.com/manual/tutorial/install-mongodb-on-ubuntu/>

2. Enable security following general guidelines at [this link](#)
3. Start MongoDB service

```
sudo service mongod start
```

4. In another prompt connect to MongoDB

```
mongo --host 127.0.0.1:27017
```

5. Create super user

```
$ use admin
$ db.createUser(
{
  user: "superAdmin",
  pwd: "[your admin password]",
  roles: [ { role: "root", db: "admin" } ]
})
```

6. Disconnect and re-connect at MongoDB as super user:

```
./connect-mongo-super-user.sh [your admin password]
```

7. Create user access (readWrite) for specific dsh database

```
$ use dsh
$ db.createUser(
{
  user: "dshuser",
  pwd: "[your password]",
  roles: [ "readWrite" ]
})
```

8. Disconnect and re-connect at MongoDB as specific user:

```
./connect-mongo.sh [your dshuser password]
```

4.2.4 RabbitMQ

Windows

1. Follow the instructions at <http://www.rabbitmq.com/install-windows.html>
2. Enable the ports mentioned at the link above at the firewall.
3. Enable the management plugin:

```
rabbitmq-plugins.bat enable rabbitmq_management
rabbitmq-service.bat stop
rabbitmq-service.bat remove
rabbitmq-service.bat install
rabbitmq-service.bat start
```

4. Test it with <http://localhost:15672/mgmt>. User: guest. Password: guest.

Linux Ubuntu 18.04 LTS

1. Follow the instructions at <https://www.rabbitmq.com/install-debian.html>
 1. As Ubuntu has a 3.5.x version it is better to download the .deb for version 3.7.x from link above
 2. Or follow the instructions at the link and add RabbitMQ Ubuntu repositories before to run the `apt-get install`.
2. Enable the management plugin:

```
sudo rabbitmq-plugins enable rabbitmq_management
service rabbitmq-server stop
service rabbitmq-server start
```

3. Test it with <http://localhost:15672/mgmt>. User: guest. Password: guest.

4.2.5 Building From Sources

1. In order to build, both MongoDB and RabbitMQ services should be running.
2. Maven development user settings should be correctly configured — see [Maven Settings for GitHub Packages](#) below, which is what lets Maven resolve the parent POM `com.mriss.mriss-parent:products`.
3. At the root dsh folder type:

```
mvn clean install
```

4.2.6 Tomcat

Windows

1. Download Tomcat 8.0.X 32-bit/64-bit Windows Service Installer at <https://tomcat.apache.org/download-80.cgi>. This will install Tomcat as a windows service.
2. Start Tomcat windows service using windows services application.
3. Look at the address: <http://localhost:8080>

Linux Ubuntu 19.04 LTS

1. Download Tomcat 8.0.x .zip or .tar.gz file at <https://tomcat.apache.org/download-80.cgi>.
2. Unpack the contents on a folder.
3. Go to the bin folder and type `./startup.sh`.
4. Look at the address: <http://localhost:8080>

5 Configuration

5.1 Maven Settings for GitHub Packages

The parent POM `com.mriss.mriss-parent:products` is published to GitHub Packages, so no build of this repository resolves until your `~/.m2/settings.xml` authenticates against that registry. Three server ids and one property are needed — the same four things that `.github/workflows/ci.yml` writes into its own `settings.xml`:

Entry	Used for
<code>server MRISS-Projects-maven-repo</code>	resolving the parent POM and other MRISS artifacts
<code>server MRISS-Projects-maven-repo-plugins</code>	resolving MRISS Maven plugins
<code>server github.com</code>	<code>maven-scm-plugin</code> , when a <code>-Ddeployment</code> build commits the regenerated <code>README.md</code>
<code>property github.personal.token</code>	<code>maven-changes-plugin</code> , which reads the closed milestone issues that go into the generated <code>README.md</code> . It is a Maven property, not a server credential — the <code>readme-generation</code> profile in <code>parent-poms</code> wires it into the plugin's <code>personalToken</code> parameter — so omitting it fails silently : <code>failOnError</code> is <code>false</code> , the issue list is never produced, and the generated <code>README.md</code> comes out with an empty Release Notes section

The credential is one and the same classic personal access token, carrying `read:packages` and nothing else, issued from an account with read access to the MRISS-Projects organisation. This repository is public, so no issue-reading scope is required on top. Paste it into all four places in your own `~/.m2/settings.xml`; it is never committed to this repository. The CI side of the same token is described under Secrets in `docs/devops/README.md`.

```

<settings>
  <servers>
    <server>
      <id>MRISS-Projects-maven-repo</id>
      <username>YOUR-GITHUB-USERNAME</username>
      <password>YOUR-PACKAGES-READ-TOKEN</password>
    </server>
    <server>
      <id>MRISS-Projects-maven-repo-plugins</id>
      <username>YOUR-GITHUB-USERNAME</username>
      <password>YOUR-PACKAGES-READ-TOKEN</password>
    </server>
    <server>
      <id>github.com</id>
      <username>YOUR-GITHUB-USERNAME</username>
      <password>YOUR-PACKAGES-READ-TOKEN</password>
    </server>
  </servers>
  <profiles>
    <profile>
      <id>github-packages</id>
      <properties>
        <github.personal.token>YOUR-PACKAGES-READ-TOKEN</github.personal.token>
      </properties>
      <repositories>
        <repository>
          <id>MRISS-Projects-maven-repo</id>
          <url>https://maven.pkg.github.com/MRISS-Projects/maven-repo</url>
          <releases><enabled>true</enabled></releases>
          <snapshots><enabled>true</enabled></snapshots>
        </repository>
      </repositories>
      <pluginRepositories>
        <pluginRepository>
          <id>MRISS-Projects-maven-repo-plugins</id>
          <url>https://maven.pkg.github.com/MRISS-Projects/maven-repo</url>
          <releases><enabled>true</enabled></releases>
          <snapshots><enabled>true</enabled></snapshots>
        </pluginRepository>
      </pluginRepositories>
    </profile>
  </profiles>
  <activeProfiles>
    <activeProfile>github-packages</activeProfile>
  </activeProfiles>
</settings>

```

5.2 MongoDB Access Properties

Edit or create the maven user settings.xml file typically at \$HOME/.m2 (or %HOMEPATH%\ .m2 at windows) folder and add a default activated profile similar to this:

```

<profile>
<id>development-properties</id>
<activation>
<activeByDefault>true</activeByDefault>
</activation>
<properties>
<mongo.host>localhost</mongo.host>
<mongo.port>27017</mongo.port>
<mongo.user>dshuser</mongo.user>
<mongo.password>[password you have configured in the steps above when installing mo
</properties>
</profile>

```

Or add the properties section at any default activated profile already present at settings.xml file.

5.3 Tomcat Admin User Configuration

Stop Tomcat if it is already started, and edit the file TOMCAT_HOME/conf/tomcat-users.xml. If the tomcat-users tag is empty or with all elements commented, add the following content inside the <tomcat-users> tag.

```

<role rolename="tomcat" />
<role rolename="manager-gui" />
<role rolename="manager-script" />
<role rolename="admin-gui" />

<user username="admin" password="[your admin password]" roles="tomcat,manager-gui,m
<user username="tomcat" password="[your tomcat user password]" roles="tomcat,manage

```

Replace the admin and tomcat's password with any desired password.

6 Usage

6.1 Running Application from Eclipse Embedded Tomcat

The module dsh-rest-api is a web application. The type tag in pom.xml file is .war. Thus the first step is to install a Tomcat (8.0.X) at <https://tomcat.apache.org/download-80.cgi>. After that, if you have Eclipse Oxygen JEE version correctly installed and configured, then is just a matter of showing the Servers view and adding a new server. At eclipse menu, follow the path: Window -> Show View -> Other -> Servers -> Server. When the Servers view opens, add a new Tomcat Server. You will need to have a Tomcat already installed at your system, since eclipse will ask for an installed Tomcat root directory. When creating a new server inside eclipse, it will show the dsh-rest-api as a potential project to be installed in that server.

After having the dsh-rest-api inside the server, configure the server startup and shutdown timeouts to something like 120s each.

Start the server and access the application swagger UI at: `http://localhost:8080/dsh-rest-api/swagger-ui.html`.

6.2 Using Application .war File on a Servlet Container

After the build, the folder `dsh-rest-api/target` should have a file named `dsh-rest-api-[version].war`. That war file can be dropped to a servlet container to be used as a web application. At this moment the server having the servlet container should be the same having MongoDB and RabbitMQ installed, up and running.

6.2.1 Using Tomcat Application Manager

Access the Tomcat's manager usually at the address `http://localhost:8080/manager/html`. The browser will ask for user and password. Enter the user and password configured at the `TOMCAT_HOME/conf/tomcat-users.xml` (see the configuration section above).

After login, at the Deploy section, fulfill the fields:

```
Context Path:  dsh-rest-api
WAR or Directory URL: [absolute path to the generated dsh-rest-api-<version>.war fi
```

You can also upload the war file from `dsh-rest-api/target` folder, using the Choose File button at the Tomcat's manager application. However, in this case, it is recommended to rename the file `dsh-rest-api-<version>.war` to just `dsh-rest-api.war` just to not have the version name associated with the web application, which will then be used to access the application at the web browser.

Start the server and access the application swagger UI at: `http://localhost:8080/dsh-rest-api/swagger-ui.html`.

6.2.2 Just Dropping Application .war File

Rename the file `dsh-rest-api/target/dsh-rest-api-<version>.war` to `dsh-rest-api.war` and drop it at Tomcat's webapps folder. Restart Tomcat if needed.

Start the server and access the application swagger UI at: `http://localhost:8080/dsh-rest-api/swagger-ui.html`.

6.3 Running Application Using Spring Boot Maven Plugin

Go to `dsh-rest-api` module root folder project, by using `cd dsh-rest-api` at the sources root, and run:

```
mvn spring-boot:run
```

Wait until the application boots up. Typically when the following output is present:

```

.
.
.
08:37:47.665 [main] INFO o.s.c.s.DefaultLifecycleProcessor - Starting beans in pha
08:37:47.665 [main] INFO s.d.s.w.p.DocumentationPluginsBootstrapper - Context refr
08:37:47.702 [main] INFO s.d.s.w.p.DocumentationPluginsBootstrapper - Found 1 cust
08:37:47.746 [main] INFO s.d.s.w.s.ApiListingReferenceScanner - Scanning for api l
08:37:47.962 [main] INFO o.a.coyote.http11.Http11NioProtocol - Initializing Protoc
08:37:47.979 [main] INFO o.a.coyote.http11.Http11NioProtocol - Starting ProtocolHa
08:37:47.984 [main] INFO o.a.tomcat.util.net.NioSelectorPool - Using a shared sele
08:37:48.016 [main] INFO o.s.b.w.e.tomcat.TomcatWebServer - Tomcat started on port
08:37:48.022 [main] INFO c.m.dsh.restapi.DshRestApplication - Started DshRestAppli
08:37:48.025 [main] INFO c.m.dsh.restapi.DshRestApplication - Main application run

```

Start the server and access the application swagger UI at: <http://localhost:8080/swagger-ui.html>.

6.4 Swagger User Interface

Swagger UI has two methods for a document resource:

- submit: uploads a PDF file and returns a token.
- status: use the token returned in the first method to ask for the document processing status. At the moment the only status would be QUEUED_FOR_INDEXING_SUCCESS.

In order to use the methods, click on the method and after in the Try it out button (firstly for the submit method).

A new form will open with the fields to fulfill. In case of submit you will need to choose a file to upload and inform its title. In case of status, you just needs to enter the the token returned by the previous submit method call.

7 Release Notes

7.1 Version 0.3.0-SNAPSHOT

#	Type	Summary	Assignee	Reporter	Updated
114	task	Release and hotfix wrappers do not supply the build properties DSH's reactor needs	null	mriiss	9/27/26
87	task	Update Maven pinned version from 3.9.9 to 3.9.16 in documentation and GitHub Actions	null	mriiss	9/26/26

123	task	[STORY] Stop passing the Mongo setup inputs to project-staging.yml	null	mriss	9/26/26
112	task	Reclassify the Spring-context tests as integration tests and pay the unit-coverage bill	null	mriss	9/26/26
117	task	Pass development_bar to the release and hotfix wrappers	null	mriss	9/25/26
111	task	[STORY] Let release.yml and hotfix.yml dispatch a release rehearsal	null	mriss	9/23/26
94	task	Make check-spec-references enforcing, or remove it	null	mriss	9/19/26
85	task	Update documentation: replace Maven 3.3.9 with 3.9.9 and standardise Java version to 17	null	mriss	9/19/26
86	task	Pin Maven 3.9.9 in all GitHub Actions workflows that invoke Maven	null	mriss	9/18/26
97	task	[STORY] Resolve the tooling orphaned by the Travis estate removal	null	mriss	9/18/26
101	task	[STORY] Fail CI when the package token cannot authenticate, not just when it is absent	null	mriss	9/18/26
103	task	Make PR review rounds repo-aware and authoritative	null	mriss	9/17/26

93	task	Remove the redundant coverage ratchet - jacoco:check at 95% is already inherited	null	mriss	9/17/26
99	task	[STORY] Standardise Maven builds on -U while the parent is a SNAPSHOT	null	mriss	9/17/26
95	bug	Use a read-only token for CI package authentication	null	mriss	9/17/26
92	task	Remove the dead Travis build estate	null	mriss	9/17/26
84	task	[FEATURE] Unify/reorg of deploy/release profiles	mriss	mriss	5/22/26
72	task	Refactor all artifactId names to be lower case to be in maven naming standards	mriss	mriss	5/22/26
83	enhancement	[FEATURE] Adapt DSH to the new parent poms version 3.8.0	mriss	mriss	4/23/26
68	wontfix	Update readme file with instructions on how to install SOLR.	mriss	mriss	4/23/26
67	wontfix	Implement configuration class and setup as a daemon.	mriss	mriss	4/23/26
66	wontfix	Configure and test OpenNLP POS filter.	mriss	mriss	4/23/26
13	wontfix	Create and test SOLR DAO	mriss	mriss	4/23/26
71	task	Adapt pom structure to new parent poms.	mriss	mriss	4/13/26

14	task	Install and get trained on SOLR tutorial	mriss	mriss	2/22/20
----	------	--	-------	-------	---------

7.2 Version 0.2.4

#	Type	Summary	Assignee	Reporter	Updated
64	bug	Test code report is being generated with 0 tests.	null	mriss	4/26/19

7.3 Version 0.2.3

#	Type	Summary	Assignee	Reporter	Updated
63	bug	Attach jacoco badge generation at verify phase is generating badge with 0%	mriss	mriss	4/26/19

7.4 Version 0.2.2

#	Type	Summary	Assignee	Reporter	Updated
61	enhancement	Add jacoco badge	mriss	mriss	4/14/19
60	enhancement	Add travis badge.	mriss	mriss	4/14/19
59	enhancement	Add jacoco coverage plugin and report.	mriss	mriss	4/12/19

7.5 Version 0.2.1

#	Type	Summary	Assignee	Reporter	Updated
58	bug	stage is being executed at master during release process	mriss	mriss	4/11/19

7.6 Version 0.2.0

#	Type	Summary	Assignee	Reporter	Updated
40	task	Add DSH to travis CI following settings from changes plugin.	mriss	mriss	4/11/19
39	task	Publish dsh site on gh-pages branch instead of another repo.	null	mriss	3/18/19
41	task	Move project from organization to the git project dsh. Change next milestone to 0.2.0	mriss	mriss	3/18/19

7.7 Version 0.0.1

#	Type	Summary	Assignee	Reporter	Updated
32	task	Configure DSH to use git as scm tool and proceed to release.	mriss	mriss	3/15/19
38	task	Configure distribution management to local nexus and test snapshot deploy with deployment profile.	mriss	mriss	3/1/19
37	task	Replace release notes and release history properties using deployment profile.	mriss	mriss	10/14/18
36	task	Replace version property ad readme and commit using deployment profile	mriss	mriss	2/19/18

35	task	Test maven site publication using github using deployment profile.	mriss	mriss	2/15/18
34	task	Configure changes plugin and changes report to use github issues and test maven site generation.	mriss	mriss	2/12/18
33	task	Configure maven scm to use git	mriss	mriss	2/12/18
10	task	Test rest API module inside tomcat server inside eclipse as a war distribution.	mriss	mriss	2/11/18
30	task	Configure Swagger	mriss	mriss	2/4/18
11	task	Create and test rest service layer:	mriss	mriss	1/16/18
23	task	Add message and error handling for RabbitMQ queue submission.	mriss	mriss	1/9/18
22	task	Create document submission workflow	mriss	mriss	1/6/18
28	task	Create document status enumeration and define workflow transition and validation class.	mriss	mriss	1/6/18
27	task	Add extra columns at the Document model class for status description and status message.	mriss	mriss	1/6/18
25	task	Test message sending exception.	mriss	mriss	1/4/18

17	task	Create web service logic to generate token and return it while starting the document storage at mongo asynchronously.	mriss	mriss	1/3/18
9	task	Create services:	mriss	mriss	12/19/17
21	task	Create mongodb storage service.	mriss	mriss	12/19/17
20	task	Update documentation with RabbitMQ installation.	mriss	mriss	12/19/17
18	task	Create logic to enqueue the mongo document id to RabbitMQ using Spring integration example app.	mriss	mriss	12/18/17
19	task	Feature/mongo dao	mriss	mriss	12/10/17
15	task	Create and test MongoDAO	null	mriss	12/10/17
16	task	Test models	mriss	mriss	12/9/17
12	task	Create dsh-test-dataset module having all PDF and HTML files used for testing.	mriss	mriss	12/8/17
8	task	* Create model for the documents with following columns:	mriss	mriss	12/7/17
7	task	* Create package structure	null	mriss	12/7/17
3	task	Create parent pom	mriss	mriss	12/7/17
5	task	Create a model module to have the model classes of keywords, sentences and documents.	mriss	mriss	12/7/17

6	task	Organize dependency management among modules.	mriss	mriss	12/7/17
2	task	Create project structure using spring boot	mriss	mriss	12/7/17
4	task	Complete project structure	mriss	mriss	12/7/17
1	task	Install RabbitMQ	mriss	mriss	11/29/17